ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



KIẾN TRÚC MÁY TÍNH (CO2008)

BÁO CÁO LAB 1
Arithmetic instructions

Sinh viên: Hoàng Đức Tài - 2510219

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

Mục lục

1 Exercise 1 **1**

2 Exercise 2 **2**

3 Exercise 3 **3**

4 Exercise 4 **4**

5 Exercise 5 **5**

6 Exercise 6 **6**

7 Exercise 7 **6**

1 Exercise 1

Tên được đọc vào buffer 100 byte trong `.data` bằng syscall 8. `$a0` là địa chỉ buffer, `$a1` là số ký tự tối đa:

```
    li $v0, 8     # syscall 8: đọc chuỗi từ bàn phím
    la $a0, name  # $a0 = địa chỉ buffer nhận chuỗi
    li $a1, 100   # $a1 = số ký tự tối đa
    syscall
```

Syscall 8 giữ lại ký tự xuống dòng ở cuối chuỗi, nên nếu in thẳng thì dấu `!` sẽ bị xuống dòng. Vòng lặp dưới duyệt từng byte bằng `lb`, dừng ở ký tự kết thúc chuỗi, và thay ký tự xuống dòng (`0x0A`) bằng `0`:

```
    la $t0, name                        # $t0 trỏ vào đầu buffer

find_newline_char:
    lb $t1, 0($t0)                      # nạp byte đang xét
    beqz $t1, print_msg                 # gặp '\0' thì hết chuỗi
    beq $t1, 10, replace_newline_char # gặp '\n' thì thay thế
    addi $t0, $t0, 1                    # sang byte kế tiếp
    j find_newline_char

replace_newline_char:
    sb $zero, 0($t0)                    # ghi 0 đè lên ký tự xuống dòng
```

Cuối cùng in ba chuỗi liên tiếp bằng syscall 4: `"Hello, "`, tên và dấu `"!"`:

```
print_msg:
    li $v0, 4          # syscall 4: in chuỗi
    la $a0, hello_msg # in "Hello, "
    syscall

    li $v0, 4          # syscall 4: in chuỗi
    la $a0, name       # in tên vừa nhập
    syscall

    li $v0, 4          # syscall 4: in chuỗi
    la $a0, excl_mark # in dấu "!"
    syscall
```

Nhập `John`:

```
Hello, John!
```

2 Exercise 2

Mảng khai báo bằng `.word 0:5` gồm 5 phần tử, mỗi phần tử 4 byte. Với chỉ số i lưu trong `$t0`, địa chỉ phần tử thứ i là base $+ 4i$, với phép nhân 4 được thay bằng dịch trái 2 bit:

```
    sll $t2, $t0, 2    # $t2 = 4i
    add $t2, $s0, $t2  # $t2 = base + 4i
    sw $t1, 0($t2)     # cất $t1 vào phần tử thứ i
```

Mỗi lời nhắc gồm 3 syscall: syscall 4 in `"Please input element "`, syscall 1 in chỉ số, syscall 11 in dấu `":"`:

```
input_element_loop:
    li $v0, 4            # syscall 4: in chuỗi
    la $a0, input_prompt # "Please input element "
    syscall

    li $v0, 1            # syscall 1: in số nguyên
    move $a0, $t0        # in chỉ số đang duyệt
    syscall

    li $v0, 11           # syscall 11: in ký tự
    li $a0, ':'          # in dấu hai chấm
    syscall
```

Chỉ số nhập vào được dùng trực tiếp để tính offset rồi nạp phần tử:

```
    li $v0, 5
    syscall

    move $t0, $v0      # đưa chỉ số vào $t0
    sll $t2, $t0, 2    # dịch trái 2 bit để nhân 4
    add $t3, $s0, $t2  # tính địa chỉ phần tử

    li $v0, 1
    lw $a0, 0($t3)
    syscall
```

Với mảng $5, 10, 15, 20, 25$, nhập chỉ số 1 rồi nhập chỉ số 4:

```
Please input element 0:5
Please input element 1:10
Please input element 2:15
Please input element 3:20
Please input element 4:25
Please enter index:1
10
```

```
Please input element 0:5
Please input element 1:10
Please input element 2:15
Please input element 3:20
Please input element 4:25
Please enter index:4
25
```

3 Exercise 3

Các chuỗi nhắc nhập và thông báo được khai báo sẵn trong vùng `.data`:

```
.data
  prompt_a: .asciiz "Insert a: "
  prompt_b: .asciiz "Insert b: "
  prompt_c: .asciiz "Insert c: "
  prompt_d: .asciiz "Insert d: "

  msg_f: .asciiz "F = "
  remainder_msg: .asciiz ", remainder "
```

Mỗi lần nhập in prompt bằng syscall 4, đọc số bằng syscall 5 rồi lưu vào thanh ghi. Dưới đây là khối nhập của a; ba biến còn lại làm tương tự với `prompt_b`–`prompt_d` và `$s1`–`$s3`:

```
  li $v0, 4         # syscall 4: in prompt
  la $a0, prompt_a
  syscall
  li $v0, 5         # syscall 5: đọc số nguyên
  syscall
  move $s0, $v0     # lưu a vào $s0
```

Tử số gồm ba thừa số: $a + 10$ dùng `addi`, $b - d$ dùng `sub`, còn $c - 2a$ thì dịch trái a 1 bit rồi trừ. Sau đó nhân dồn lại:

```
  addi $t0, $s0, 10 # $t0 = a + 10
  sub $t1, $s1, $s3 # $t1 = b - d

  sll $t2, $s0, 1   # $t2 = 2a
  sub $t2, $s2, $t2 # $t2 = c - 2a

  add $t3, $s0, $s1 # $t3 = a + b
  add $t3, $t3, $s2 # $t3 = a + b + c

  mul $t0, $t0, $t1 # tử = (a + 10)(b - d)
  mul $t0, $t0, $t2 # nhân tiếp với (c - 2a)
```

```
    div $t0, $t3       # chia tử cho mẫu
    mflo $t0           # $t0 = thương
    mfhi $t1           # $t1 = số dư
```

Với $a = 3$, $b = 4$, $c = 5$, $d = 6$: tử số là $13 \times (-2) \times (-1) = 26$, mẫu số là 12, nên $F = 2$ dư 2.

```
Insert a: 3
Insert b: 4
Insert c: 5
Insert d: 6
F = 2, remainder 2
```

4 Exercise 4

Bước kiểm tra dùng `sltiu`, tức so sánh không dấu với 16. Số âm ở dạng unsigned là một số không dấu rất lớn nên bị loại bởi cùng một lệnh với các số ≥ 16. Nếu nhập sai thì chương trình quay về nhãn `input` để nhập lại:

```
input:
  li $v0, 4          # syscall 4: in prompt
  la $a0, prompt
  syscall

  li $v0, 5          # syscall 5: đọc số nguyên
  syscall

  sltiu $t0, $v0, 16 # 1 nếu 0 <= n < 16, so sánh không dấu
  beq $t0, 0, input  # ngoài khoảng thì nhập lại
```

Phần in dùng `srlv` để dịch phải theo số bit lấy từ thanh ghi. Bộ đếm chạy từ 3 xuống 0, tương ứng với 4 bit từ trọng số 8 đến 1. Mỗi vòng dịch phải, dùng `andi` lấy bit, cộng `'0'` rồi in bằng syscall 11.

```
    move $s0, $v0        # lưu n vào $s0 trước khi in
    li $t0, 3            # bắt đầu từ bit 3, ứng với trọng số 8

    li $v0, 4            # syscall 4: in chuỗi
    la $a0, output_msg   # "Its binary form is: "
    syscall

print_loop:
    srlv $t1, $s0, $t0   # đưa bit thứ $t0 xuống vị trí 0
    andi $t1, $t1, 1     # lấy đúng bit đó
```

```
    addi $t1, $t1, '0'    # đổi sang mã ASCII

    li $v0, 11            # syscall 11: in ký tự
    move $a0, $t1
    syscall

    subi $t0, $t0, 1      # lùi sang bit thấp hơn
    bgez $t0, print_loop # còn bit thì lặp
```

Nhập 20 bị từ chối, sau đó nhập 10:

```
Please enter a positive integer less than 16: 20
Please enter a positive integer less than 16: 10
Its binary form is: 1010
```

5 Exercise 5

Mảng được khai báo bằng `.word` gồm 10 phần tử lưu địa chỉ vào `$s0`, cùng biến `array_size` lưu độ dài mảng vào `$s1`.

```
.data
  array: .word 1, 2, 5, 8, 12, 44, 3, 9, 0, 10
  array_size: .word 10
```

Bộ đếm `$t0` chạy từ 1 đến 10, nên chỉ số cần in là `array_size` − `$t0`. Chỉ số được đổi sang offset byte bằng `sll` 2:

```
print_reverse_loop:
  sub $t1, $s1, $t0 # chỉ số = array_size - bộ đếm
  sll $t1, $t1, 2   # đổi chỉ số sang offset byte
  add $t1, $s0, $t1 # $t1 = địa chỉ phần tử

  li $v0, 1         # syscall 1: in số nguyên
  lw $a0, 0($t1)    # nạp giá trị phần tử
  syscall
```

Dấu phẩy được in sau mỗi phần tử, trừ phần tử cuối. Khi bộ đếm bằng `array_size` thì nhảy tới `exit` trước khi in `", "`, nên kết quả không có dấu phẩy thừa:

```
  beq $t0, $s1, exit   # phần tử cuối thì không in dấu phẩy

  li $v0, 4            # syscall 4: in chuỗi
  la $a0, comma        # ", "
  syscall

  addi $t0, $t0, 1     # sang phần tử kế
```

```
    j print_reverse_loop
```

Chương trình in ra:

```
10, 0, 9, 3, 44, 12, 8, 5, 2, 1
```

6 Exercise 6

a. Dung lượng tối thiểu của một frame

Số pixel của một frame là $1280 \times 1024 = 1\,310\,720$. Mỗi pixel dùng $3 \times 8 = 24$ bit, tức 3 byte, nên dung lượng tối thiểu là:

$$1\,310\,720 \times 3 = 3\,932\,160 \text{ byte} \approx 3{,}75 \text{ MiB}$$

b. Thời gian truyền

Một frame có $3\,932\,160 \times 8 = 31\,457\,280$ bit. Với tốc độ 100 Mbit/s:

$$t = \frac{31\,457\,280}{100 \times 10^6} \approx 0{,}3146 \text{ s} \approx 314{,}6 \text{ ms}$$

7 Exercise 7

Bộ xử lý	f	CPI
P1	3,0 GHz	1,5
P2	2,5 GHz	1,0
P3	4,0 GHz	2,2

a. Bộ xử lý nào có IPS cao nhất

Số lệnh thực hiện trong một giây là IPS $= f/\text{CPI}$:

$$\text{IPS}_{P1} = \frac{3 \times 10^9}{1{,}5} = 2 \times 10^9$$
$$\text{IPS}_{P2} = \frac{2{,}5 \times 10^9}{1} = 2{,}5 \times 10^9$$
$$\text{IPS}_{P3} = \frac{4 \times 10^9}{2{,}2} \approx 1{,}82 \times 10^9$$

P2 có IPS cao nhất, dù xung nhịp của nó thấp hơn P1 và P3.

b. Số lệnh và số chu kỳ trong 10 giây

Với thời gian $t = 10$ s:

Bộ xử lý	Số chu kỳ $= f \times t$	Số lệnh $=$ IPS $\times\, t$
P1	$3{,}0 \times 10^{10}$	$2{,}00 \times 10^{10}$
P2	$2{,}5 \times 10^{10}$	$2{,}50 \times 10^{10}$
P3	$4{,}0 \times 10^{10}$	$1{,}82 \times 10^{10}$

c. Xung nhịp cần thiết để giảm 30% thời gian thực thi

Gọi f và f' là xung nhịp cũ và mới. Thời gian thực thi là $t = \dfrac{IC \times \text{CPI}}{f}$. Khi thời gian giảm 30% và CPI tăng 20%:

$$0{,}7\,t = \frac{IC \times 1{,}2\,\text{CPI}}{f'} \implies f' = \frac{1{,}2}{0{,}7}\,f \approx 1{,}714\,f$$

Bộ xử lý	f hiện tại	f' cần thiết ($\approx 1{,}714\,f$)
P1	3,0 GHz	5,14 GHz
P2	2,5 GHz	4,29 GHz
P3	4,0 GHz	6,86 GHz